

Week 5, Part 2: Gestures and State

Dr M AOUDE

Lecture notes

Objectives. This is the second part of week 5. By the end you will be able to explain the two levels of the Flutter gesture system (pointers and gestures); react to a tap, a double tap, a long press, a drag, a pan and a pinch with `GestureDetector`; explain who receives a touch when several detectors overlap; tell *ephemeral state* from *app state*; share state by lifting it up; and share it with a `ChangeNotifier` and the `provider` package.

The **programs** of this part are not printed here: they are in the project **examples**, inside `code/week05` (section 14). Each section quotes only the few lines that matter. Every program starts with a comment that says what it teaches and what result to expect, and automated tests check that it does what is described. The first part of the week, *Creating Interactive Applications*, is a separate document.

Gestures

1 What is a gesture?

A **gesture** is any physical movement that the user makes to control a touch-based device. We use them all day: we unlock a phone by swiping a finger over the screen, we tap a button, we press and hold an icon to move it, we pinch a photo to zoom. Flutter lets an app react to all of them.

Flutter separates its gesture system into **two levels**:

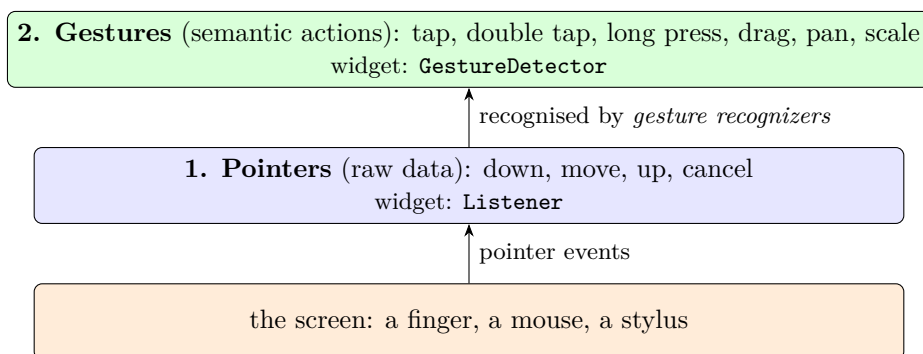


Figure 1: The two levels of the gesture system. The higher level is built from the lower one.

2 Level 1: pointers

The first level is made of **pointer events**. They are the raw data about the user's interaction: *where* the pointers (touches, mouse, stylus) are on the screen and *how they move*. The widget that listens to them is `Listener`. Four types of events are sent to a pointer:

Callback of Listener	Sent when
<code>onPointerDown</code>	the pointer touches the screen, at a position (<code>PointerDownEvent</code>)
<code>onPointerMove</code>	the pointer moves from one position to another (<code>PointerMoveEvent</code>)
<code>onPointerUp</code>	the pointer leaves the screen (<code>PointerUpEvent</code>)
<code>onPointerCancel</code>	the interaction is interrupted, for example when the system takes over the touch (<code>PointerCancelEvent</code>)

Each event gives its `position` (on the whole screen), its `localPosition` (inside the widget that listens), the `delta` of a move, and an identifier of the pointer, so that two fingers can be told apart.

```

1 Listener(
2   behavior: HitTestBehavior.opaque,
3   onPointerDown: (e) => _record('PointerDownEvent', e, () => _down++),
4   onPointerMove: (e) => _record('PointerMoveEvent', e, () => _move++),
5   onPointerUp: (e) => _record('PointerUpEvent', e, () => _up++),
6   onPointerCancel: (e) =>
7     _record('PointerCancelEvent', e, () => _cancel++),

```

Two facts to remember: `Listener` sees *every* pointer event, and Flutter provides no way to cancel or stop the delivery of pointer events. Example `00_pointer_events` shows the last event, the position and the four counters on the screen. In an app we rarely need this level: we use the next one.

3 Level 2: gestures

The second level represents **semantic actions**, such as tap, drag and scale, that Flutter detects from many pointer events. It also sends events about the *life* of a gesture: start, update, end. The most common gestures are:

Gesture	Callbacks	What it is
Tap	<code>onTapDown</code> , <code>onTapUp</code> , <code>onTap</code> , <code>onTapCancel</code>	touch the screen briefly and release. <code>onTap</code> means the tap is complete; <code>onTapCancel</code> means it did not happen (the finger moved away, another gesture won)
Double tap	<code>onDoubleTap</code>	two taps in a short time
Long press	<code>onLongPress</code>	touch and hold in one place for a longer time (about half a second)
Horizontal drag	<code>onHorizontalDragStart</code> , <code>onHorizontalDragUpdate</code> , <code>onHorizontalDragEnd</code>	touch, move left or right, release
Vertical drag	<code>onVerticalDragStart</code> , <code>onVerticalDragUpdate</code> , <code>onVerticalDragEnd</code>	touch, move up or down, release
Pan	<code>onPanStart</code> , <code>onPanUpdate</code> , <code>onPanEnd</code>	touch and move in <i>any</i> direction, without lifting the finger
Scale (pinch)	<code>onScaleStart</code> , <code>onScaleUpdate</code> , <code>onScaleEnd</code>	zoom in or out by moving two fingers apart or together; also gives the rotation

Every callback receives a **details** object with the data of the gesture:

Details object	Useful fields
TapDownDetails, TapUpDetails	globalPosition, localPosition: where the finger touched
DragUpdateDetails	delta: how far the finger moved since the previous call (delta.dx horizontally, delta.dy vertically); primaryDelta for a one-direction drag
DragEndDetails	velocity, primaryVelocity: the speed of the finger when it was lifted, in pixels per second
ScaleUpdateDetails	scale: the zoom since the start of the gesture (1.0 = no change); rotation: the angle in radians; focalPoint

4 The GestureDetector widget

GestureDetector gives Flutter great support for all these gestures. It is a **non-visual** widget: it draws nothing; it *wraps* a child, listens to the pointer events on that child, and calls your functions. Its constructor has one parameter for each gesture callback, and it looks for the gestures whose callback is **not null**.

```

1 GestureDetector(
2   onTapDown: (details) => _happened('onTapDown'),
3   onTapUp: (details) => _happened('onTapUp'),
4   onTap: () => _happened('onTap'),
5   onTapCancel: () => _happened('onTapCancel'),
6   onDoubleTap: () => _happened('onDoubleTap'),
7   onLongPress: () => _happened('onLongPress'),

```

It is used like this: a box that reacts to a tap (the code of the book, updated):

```

1 GestureDetector(
2   onTap: () => setState(() => _clicks++),
3   child: Container(
4     height: 62, width: 110,
5     decoration: BoxDecoration(color: Colors.blueGrey, borderRadius: BorderRadius
6       .circular(18)),
7     child: const Center(child: Text('Click me')),
8   ),
9 )

```

The *child* decides what is seen and how big the touch area is. Example 01_tap_gestures counts the six events of a tap, a double tap and a long press.

Where does the detector react? behavior

Not every part of the box receives the touch. The **behavior** parameter of GestureDetector says which:

HitTestBehavior	Meaning
deferToChild	(the default) the detector reacts only where its child is <i>hit</i> : on a text, a color, an icon. An empty SizedBox draws nothing, so the touch goes through it
opaque	the whole area of the detector receives the touch, drawn or not, and it blocks the widgets behind
translucent	the whole area receives the touch, and the widgets behind it receive it too

Gestures compete: timing

The detector does not know yet what the finger is doing, so its recognizers wait for each other. This has visible effects, checked by the tests of example 01:

- When `onDoubleTap` is set, Flutter must see whether a second tap follows: the *whole* tap (`onTapDown`, `onTapUp` and `onTap`) arrives about 0.3 s after the finger is lifted. If you do not need a double tap, do not set `onDoubleTap`: a tap is then instant.
- A *long press* wins against the tap: `onLongPress` is called, and the tap is cancelled (`onTapCancel`).
- If the finger moves away before it is lifted, the tap is cancelled too.
- A *pan* and a *scale* cannot be set on the same detector: *scale is a superset of pan* (with one finger it acts as a pan), and Flutter stops with an assertion. The same happens with a pan together with *both* a horizontal and a vertical drag, because the two drags would catch every movement and the pan would never be called. When you need different gestures, give each its own `GestureDetector`.

5 Drag, pan and pinch

To make a widget follow the finger, we do three things: keep its position in the **state**, add the **delta** of each update to it *inside* `setState`, and draw the widget at that position. `clamp` keeps it inside its area. Example 02_drag_gestures moves two squares, one along a horizontal track and one along a vertical track:

```
1 onHorizontalDragStart: (details) {
2   setState(() => _last = 'onHorizontalDragStart');
3 },
4 onHorizontalDragUpdate: (details) {
5   setState(() {
6     _x = (_x + details.delta.dx).clamp(0, _track).toDouble();
7     _last = 'onHorizontalDragUpdate';
8   });
9 },
10 onHorizontalDragEnd: (details) {
11   setState(() => _last = 'onHorizontalDragEnd');
12 },
```

A horizontal drag uses only the horizontal part of the movement: dragging diagonally moves the red square only sideways. The square is placed with a `Positioned` widget inside a `Stack` (week 4, part 2), whose `left` is the state `_x`.

A **pan** moves something in any direction: its `delta` has both `dx` and `dy`. Example 03_pan_ball moves a ball inside a square:

```
1 onPanUpdate: (details) {
2   setState(() {
3     // delta has dx AND dy: the ball moves in both.
4     final next = _position + details.delta;
5     _position = Offset(
6       next.dx.clamp(0, _field - _ball).toDouble(),
7       next.dy.clamp(0, _field - _ball).toDouble(),
8     );
9   });
10 },
```

A **pinch** is a *scale* gesture. Its `scale` always starts again at 1.0 for a new gesture, so we remember the size at `onScaleStart` and multiply (example 04_pinch_scale):

```
1 onScaleStart: (details) {
2   _startScale = _scale;
```

```

3  _startRotation = _rotation;
4  },
5  onScaleUpdate: (details) {
6    setState(() {
7      _scale = (_startScale * details.scale)
8        .clamp(0.5, 3.0)
9        .toDouble();
10     _rotation = _startRotation + details.rotation;
11   });
12 },

```

`Transform.scale` and `Transform.rotate` change how a widget is drawn without changing the layout around it.

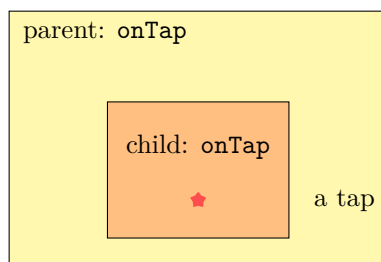
Ready-made widgets. Before you write a gesture yourself, check that Flutter does not already have the widget:

<code>InkWell</code>	a tap with the Material “ripple” effect (it needs a Material ancestor, such as a Scaffold)
<code>IconButton</code> , <code>ElevatedButton</code>	buttons: they already handle taps
<code>Dismissible</code>	swipe an item away (a list row that you delete by sliding it)
<code>Draggable</code> , <code>DragTarget</code>	drag a widget and drop it on another
<code>InteractiveViewer</code>	zoom, pan and rotate an image or a map, with the fingers
<code>Slider</code> , <code>Switch</code> , <code>Checkbox</code>	controls that are dragged or tapped

6 Several detectors at the same place: the arena

Often two detectors are at the same place: a big *parent* and a small *child* inside it, both with `onTap`. When the user taps the child, who receives the tap?

Flutter organises an **arena**. Every detector under the finger enters the competition, and **only one wins** the gesture. For a tap on the child, the *innermost* detector wins: the child gets `onTap` and the parent gets nothing. This is the default, and it is usually what we want: a button inside a card must not also trigger the card.



default: the child wins
the parent gets nothing

RawGestureDetector with
`AllowMultipleGestureRecognizer`:
both get the tap

Figure 2: A tap on the child.

If you *really* want both to react, you must change the rules with a `RawGestureDetector`, which takes a map of “gesture factories” and lets you use your own recognizer. `AllowMultipleGestureRecognizer` is a `TapGestureRecognizer` that, when it *loses* the arena, accepts the gesture anyway:

```

1  class AllowMultipleGestureRecognizer extends TapGestureRecognizer {
2    @override
3    void rejectGesture(int pointer) {
4      acceptGesture(pointer);

```

```

5 }
6 }

```

The factory has two functions: one *creates* the recognizer, the other *sets its callbacks*.

```

1 Widget _multipleTap({required VoidCallback onTap, required Widget child}) {
2   return RawGestureDetector(
3     behavior: HitTestBehavior.opaque,
4     gestures: {
5       AllowMultipleGestureRecognizer:
6         GestureRecognizerFactoryWithHandlers<AllowMultipleGestureRecognizer>() => AllowMultipleGestureRecognizer(), // creates the recognizer
7       (instance) {
8         instance.onTap = onTap; // sets its callbacks
9       },
10    },
11  ),
12 },
13 child: child,
14 );
15 }

```

Example 05_gesture_arena shows both behaviours side by side, with counters on the screen: a tap on the orange child adds 1 to the child counter only on the left, and to both counters on the right. (A real app prints such messages to the console with `print`, but you cannot see the console of a phone; a counter on the screen is easier.)

State

7 What is state?

In week 4 we saw that a widget is **stateless** or **stateful**. A stateless widget has no internal state: it cannot change once it is created, and to show something else the parent must build a new widget. A stateful widget keeps a **state** and can change during its life without being created again.

The **state** of an app is the information that can be read when a widget is built and that may change while the app runs. Flutter is **declarative**: we do not modify the screen, we *describe* it from the current state. When the state changes, we tell Flutter, and Flutter builds the described interface again:

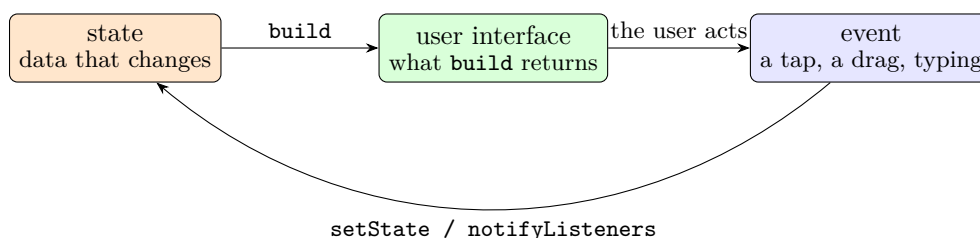


Figure 3: The interface is a function of the state: $UI = f(state)$.

Think of a list of products in an app. When a new product is added, the list on the screen must be updated. The list is the *state*; the screen is a picture of it. State management is deciding *where the state lives*, and *how the widgets that show it are told that it changed*.

Flutter separates state into two conceptual categories:

	Ephemeral state	App state
Also called	UI state, local state	shared state, application state
Belongs to	one widget	many widgets, many screens
Lives	as long as that widget	across screens, sometimes across sessions
Examples	a checkbox, the text of a field, the open tab, the page of a carousel	user preferences, login information, the shopping cart, notifications, read/unread articles
Managed with	<code>setState</code> in the <code>State</code> class	lifting the state up, <code>ChangeNotifier</code> , provider

8 Ephemeral state

Ephemeral state is state contained in a *single widget*. We do not need a state-management technique for it: it is a private field of the `State` class, changed with `setState`. A `TextField` is a good example (week 5, part 1). Here is a heart button; the state is a `bool`:

```

1 class _LikeButtonState extends State<LikeButton> {
2   bool _liked = false; // the state (it changes)
3
4   @override
5   Widget build(BuildContext context) {
6     return Row(
7       mainAxisAlignment: MainAxisAlignment.min,
8       children: [
9         Text(widget.label), // widget.xxx reads the configuration
10        IconButton(
11          icon: Icon(
12            _liked ? Icons.favorite : Icons.favorite_border,
13            color: _liked ? Colors.red : null,
14          ),
15          onPressed: () {
16            setState(() {
17              _liked = !_liked;
18            });
19          },
20        ),
21      ],
22    );
23  }

```

- `_liked` is the *ephemeral state*. Only this `State` class can read it and change it. Its constructor parameter `label` is the widget's *configuration*: it never changes, and the state reads it with `widget.label`.
- `setState` runs the function, then Flutter calls `build` again, and the widget is replaced by a new one with the new value.
- **Every widget object has its own state.** Example 06_ephemeral_state shows three `LikeButton` widgets: tapping one heart does not change the other two. It also has the classic button whose label is its state, Rami then Lina.

9 App state: lifting the state up

Some state is needed by *several* widgets. In example 07_lifting_state_up, the number of items in a shopping cart is shown in the app bar, in the body and on the **Add** button, and

the button changes it. A widget cannot read the private state of another widget. The simplest solution is to **lift the state up**: keep it in the closest common ancestor, here `ShopPage`.

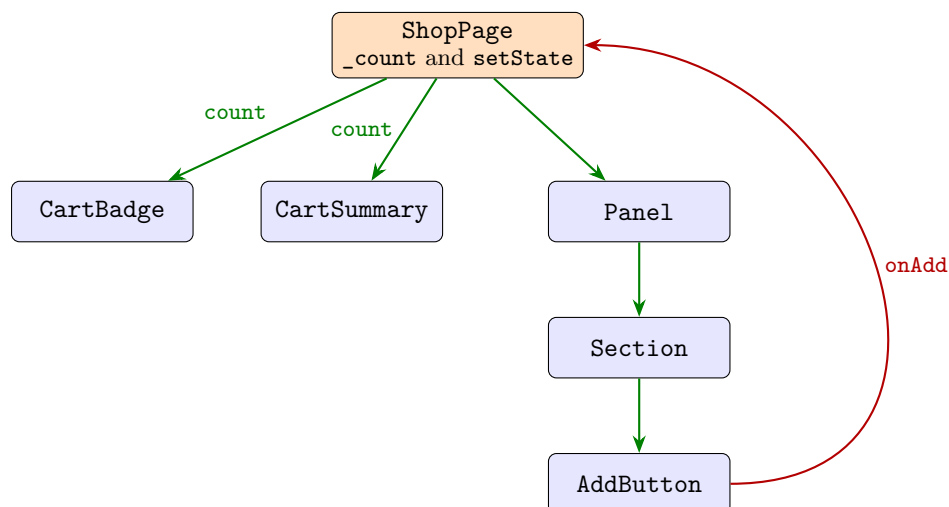


Figure 4: The data flows **down** as parameters, the events flow **up** as callbacks.

- the **data flows down**, as constructor parameters (`count`);
- the **events flow up**, as callback functions (`onAdd`), which the ancestor implements with `setState`.

It works well, but it has a cost: `AddButton` is three levels below `ShopPage`, so `Panel` and `Section` must receive `count` and `onAdd` and pass them on *without using them*. In a big app this becomes heavy. The next two sections remove the problem.

10 ChangeNotifier

Instead of passing the state through the widgets, we put it in a separate **model** class, and the widgets that need it *listen* to it. In Flutter the basic tool is **ChangeNotifier**: a simple class that **notifies its listeners** when something changes. It is part of Flutter itself, and it is easy to understand, to implement and to optimise for a small number of listeners.

```

1 class CounterModel with ChangeNotifier {
2   int _value = 0;
3
4   int get value => _value;
5
6   void increment() {
7     _value++;
8     notifyListeners(); // tell the listeners that the data changed
9   }
10
11   void decrement() {
12     _value--;
13     notifyListeners();
14   }
15 }
  
```

- The model keeps the data in a private field and changes it in methods (`increment`, `decrement`).
- After each change it calls `notifyListeners()`. This is the only method a **ChangeNotifier** model needs to call. It tells the listeners: “I changed: draw yourselves again”. If you forget it, the data changes but the screen does not.

A widget listens with a `ListenableBuilder`: its `builder` function is called again each time the model notifies.

```

1 ListenableBuilder(
2   listenable: _model,
3   builder: (context, child) {
4     _textBuilds++;
5     return Column(
6       children: [
7         Text('Counter: ${_model.value}',
8           style: const TextStyle(fontSize: 24)),
9         Text('Text built: $_textBuilds time(s)'),
10      ],
11    );
12  },
13 ),

```

In example `08_change_notifier` the page has **no** `setState` at all: the buttons only call `model.increment()`. Two counters on the screen prove that *only the builder* is rebuilt: after three taps the text was built four times, and the page still once. (Counting the builds inside `build` is only for teaching: a real `build` method must have no side effects.) The model is created once, in `initState`, and released in `dispose`.

11 The provider package

`ListenableBuilder` still needs the model to be *passed* to the widget. The **provider** package makes a model available to the *whole* widget tree below a point, so a widget anywhere can find it without receiving it as a parameter. It is a third-party library, written by the Flutter community and recommended by the Flutter team for simple app state. Add it to the project, then download it:

```

flutter pub add provider
flutter pub get

```

Then import `'package:provider/provider.dart';`. Three ideas are enough to start.

`ChangeNotifier`, `ChangeNotifierProvider`, `Consumer`

- **ChangeNotifier**: the model, as in the previous section. In example `09_provider_cart` the `CartModel` keeps a list of items and calls `notifyListeners()` in `add`, `remove` and `clear`:

```

1 class CartModel extends ChangeNotifier {
2   final List<Item> _items = [];
3
4   List<Item> get items => List.unmodifiable(_items);
5
6   double get total => _items.fold(0, (sum, item) => sum + item.price);
7
8   void add(Item item) {
9     _items.add(item);
10    notifyListeners();
11  }
12
13  void remove(Item item) {
14    _items.remove(item);
15    notifyListeners();
16  }
17
18  void clear() {
19    _items.clear();

```

```

20     notifyListeners();
21   }
22 }

```

- **ChangeNotifierProvider**: a widget that *creates* a model and gives it to all the widgets below it. It builds the model only when it is needed, and calls `dispose()` on it when it is no longer needed. Its `create` function makes the model.
- **Consumer**: a widget that finds the model and rebuilds its `builder` each time the model notifies. The builder receives three values: the `context`, the `model`, and a `child`, a part of the tree that does not depend on the model: it is built once and passed back, which saves work.

MultiProvider

To give *several* models we do not nest the providers one inside another: a `MultiProvider` takes a list of them. Here the app has two models, the cart and the user's settings, and the provider is placed **above** the `MaterialApp`, so that every screen, also the ones opened with `Navigator.push`, sees the same cart:

```

1  return MultiProvider(
2    providers: [
3      ChangeNotifierProvider(create: (_) => CartModel()),
4      ChangeNotifierProvider(create: (_) => SettingsModel()),
5    ],
6    // The theme depends on the settings: a Consumer around MaterialApp.
7    child: Consumer<SettingsModel>(
8      builder: (context, settings, child) {
9        return MaterialApp(
10          debugShowCheckedModeBanner: false,
11          theme: settings.dark ? ThemeData.dark() : ThemeData.light(),
12          home: const CatalogPage(),
13        );
14      },
15    ),
16  );

```

Finding a model from a BuildContext

Besides `Consumer`, provider adds three methods to `BuildContext`:

Method	Effect	Use it
<code>context.watch<T>()</code>	returns the model <i>and rebuilds</i> the widget when it notifies	in build
<code>context.read<T>()</code>	returns the model, <i>without</i> rebuilding	in callbacks: <code>onPressed</code> , <code>onTap</code>
<code>context.select<T, R>(f)</code>	rebuilds only when the part selected by <code>f</code> changes	in build, to rebuild less

In the example, the cart icon uses `select`: it is rebuilt only when the *number* of items changes, not for every change of the cart. A button of the catalog only *reads* the model, in its callback:

```

1  final count = context.select<CartModel, int>((cart) => cart.items.length);
2  ...
3  onPressed: () => context.read<CartModel>().add(item),

```

Rules of thumb. Put a `Consumer` (or a `select`) as **deep in the tree as possible**: only the small part that needs the data is rebuilt, not the whole page. Read with `watch` in `build`, with `read` in callbacks. Never change a model from `build`.

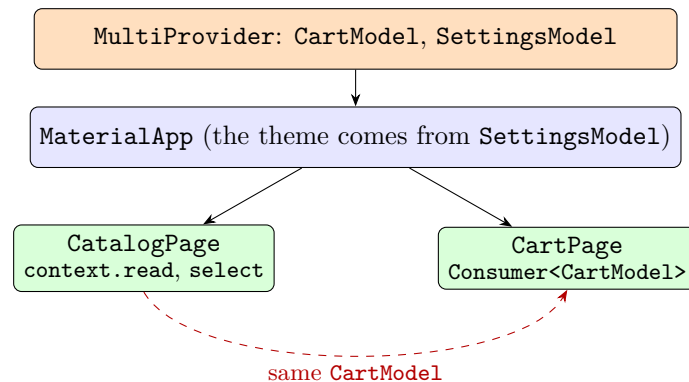


Figure 5: The provider is above the `MaterialApp`: both pages share one cart.

Other solutions

`provider` is not the only one. Larger apps often use `Riverpod` (from the same author), `Bloc`, or `Redux`-like packages; they solve the same problem with more structure. The ideas of this section (a model outside the widgets, notifying its listeners, and widgets that rebuild when it changes) are the same in all of them.

Which one to choose?

The state is...	Use
needed by one widget only	<code>setState</code> in its <code>State</code> class (ephemeral state)
needed by a few widgets that are close in the tree	lift it up to the common ancestor: parameters down, callbacks up
needed by many widgets, or many screens, or must outlive a screen	a <code>ChangeNotifier</code> model, given with <code>provider</code>
a single value	a <code>ValueNotifier<T></code> (a <code>ChangeNotifier</code> for one value) with a <code>ValueListenableBuilder</code>

12 What we changed from the printed text

The chapter that this part follows was written for older versions of Flutter. The programs of this course use the current API:

In the printed text	In our programs
<code>RaisedButton</code>	<code>ElevatedButton</code> (week 4, part 3)
<code>ChangeNotifierProvider(builder: ...)</code>	<code>ChangeNotifierProvider(create: ...)</code> : the <code>builder</code> parameter was renamed
<code>new HomePageState(), new Scaffold(...)</code>	no <code>new</code> : Dart does not need it
<code>MyHomePageState</code> <code>createState() => new HomePageState()</code>	<code>State<HomePage> createState() => _HomePageState()</code> : the names must match, and the return type is <code>State<HomePage></code>
the list of vertical drag events repeats <code>onVerticalDragStart</code> three times	<code>onVerticalDragStart</code> , <code>onVerticalDragUpdate</code> , <code>onVerticalDragEnd</code>
<code>notifyListener()</code>	<code>notifyListeners()</code> , with an <code>s</code>
<code>print('Box-Clicked')</code>	the result is shown on the screen (a counter, a text), because the console of a phone is not visible
<code>ThemeData(primarySwatch: ...)</code>	a theme built from a seed color or <code>ThemeData.light()</code>

13 Common errors

Message or symptom	Cause	Fix
Nothing happens when the box is tapped	the box is an empty <code>SizedBox</code> (or has no color): a detector reacts only where its child is hit	give it a color or a child, or behavior : <code>HitTestBehavior.opaque</code>
<code>onTap</code> arrives late	<code>onDoubleTap</code> is set on the same detector	remove <code>onDoubleTap</code> if you do not need it
<i>Incorrect GestureDetector arguments</i> (assertion)	<code>onPanUpdate</code> together with <code>onScaleUpdate</code> (or a pan together with both a horizontal and a vertical drag)	keep only one, or use two detectors
The parent does not get the tap of its child	the child wins the arena	that is the default; use <code>RawGestureDetector</code> to allow both
The widget moves but jumps back or does not move	the position was changed without <code>setState</code> , or is not used by the widget	change the position in <code>setState</code> , and draw at that position
The screen does not change after <code>model.increment()</code>	<code>notifyListeners()</code> was forgotten, or nobody listens	call it in every method that changes the data; use a <code>Consumer</code>
<i>Could not find the correct Provider<CartModel> above this widget</i> (<code>ProviderNotFoundException</code>)	the provider is not an <i>ancestor</i> of the widget: it is below it, or in another branch of the tree	put the provider above the <code>MaterialApp</code> , or above every widget that needs it
The whole page is rebuilt for each change	<code>context.watch</code> high in the tree	use a <code>Consumer</code> or <code>select</code> lower in the tree

14 The programs

The code is **not** printed in this document. It is in the folder `code/week05/examples`, a small Flutter project with one runnable app per file. See the guide *Running the Flutter apps in VS Code* to prepare the tools; then, from the folder `examples`:

```
flutter pub get          (this one also downloads the provider package)
flutter test
flutter create .         (one time only)
flutter run -t lib/01_tap_gestures.dart
```

File	What you see and try	Section
00_pointer_events	a square; the last pointer event, the position and four counters: down, move, up, cancel	2
01_tap_gestures	a box and six counters: tap down, up, tap, cancel, double tap, long press. Notice the 0.3 s delay of a tap	3, 4
02_drag_gestures	a red square that slides along a horizontal track, a green one along a vertical track	5
03_pan_ball	an orange ball that you move in any direction inside a square	5
04_pinch_scale	a square that you zoom (and rotate) with two fingers; the size is kept	5
05_gesture_arena	two yellow squares with an orange square inside: on the left the child wins the tap, on the right both react	6
06_ephemeral_state	a button whose label is its state, and three hearts, each with its own state	8
07_lifting_state_up	the number of items in a cart, shown in three places, and changed by a button three levels down	9
08_change_notifier	a counter with a ChangeNotifier and no setState ; two lines prove that only the text is rebuilt	10
09_provider_cart	a catalog and a cart page that share a cart, with a badge, a total and a dark theme, using provider	11

On a computer, the mouse acts as one finger; the pinch of example 04 needs an emulator or a phone (in the Android emulator, hold **Ctrl** to show two fingers).

15 Exercises

1. In `01_tap_gestures`, remove `onDoubleTap`. What changes for the tap? Put it back and measure the delay.
2. In `00_pointer_events`, remove `behavior: HitTestBehavior.opaque` and replace the `Container` by an empty `SizedBox`. Does it still receive events? Why?
3. In `02_drag_gestures`, add `onHorizontalDragEnd` code that uses `details.primaryVelocity` to display “fast” or “slow”.
4. Make the ball of `03_pan_ball` change color while it is moved (use the `_panning` state).
5. In `04_pinch_scale`, change the limits 0.5 and 3.0. Then replace the pinch by an `InteractiveViewer` around an `Image`.
6. In `05_gesture_arena`, add a third level (a `GestureDetector` inside the orange square). Who wins in the default demo?
7. In `06_ephemeral_state`, add a fourth `LikeButton`. Does it need any change of the other classes? Why?
8. In `07_lifting_state_up`, add a **Remove** button next to **Add**. Count how many widgets must receive a new parameter.

9. In `08_change_notifier`, add a second model, `NameModel`, and a text field that changes it. Which widgets are rebuilt when you type?
10. In `09_provider_cart`, add a **quantity** to the items, or a **Favorites** model given by a third provider in the `MultiProvider`.

16 Summary

Idea	In one sentence
Two levels	pointers (raw events: down, move, up, cancel; <code>Listener</code>) and gestures (tap, drag, scale...; <code>GestureDetector</code>)
<code>GestureDetector</code>	an invisible widget that calls the callbacks that are not null; the child decides what can be hit; <code>behavior</code> changes it
Tap	<code>onTapDown</code> , <code>onTapUp</code> , <code>onTap</code> , <code>onTapCancel</code> ; a tap is delayed if <code>onDoubleTap</code> is set
Drag and pan	<code>onHorizontalDrag*</code> , <code>onVerticalDrag*</code> , <code>onPan*</code> : add <code>details.delta</code> to the position, in <code>setState</code>
Scale	<code>details.scale</code> restarts at 1.0: multiply the size kept at <code>onScaleStart</code>
Arena	several detectors compete and one wins, the innermost; <code>RawGestureDetector</code> allows several
State	the data that changes; the UI is a function of it
Ephemeral state	belongs to one widget: a field of its <code>State</code> , changed with <code>setState</code>
App state	shared by many widgets or screens
Lifting state up	keep it in the common ancestor: data down as parameters, events up as callbacks
<code>ChangeNotifier</code>	a model that calls <code>notifyListeners()</code> ; <code>ListenableBuilder</code> rebuilds only what listens
provider	<code>ChangeNotifierProvider(create:)</code> , <code>MultiProvider</code> , <code>Consumer</code> , <code>context.watch</code> / <code>read</code> / <code>select</code> ; place it above the widgets that use it

This document is maintained by Dr. Mohamad AOUE

Last updated: 2026-09-21